

DeepPersona 코드 학습교재

모듈 02 — 속성 트리(Taxonomy, Stage 1)와 데이터 구조

Jin Sam Cho · cho@kaist.ac.kr

September 13, 2026

원저: Wang et al., *DeepPersona* (arXiv:2511.07338) · 논문 §3.1

이 모듈의 목표

한눈에 페르소나의 "빈 슬롯 창고"인 **속성 트리**가 파일 안에서 어떤 모양이며, 어떤 코드가 그걸 만들었는지 본다.

얻는 것

- 트리 = 중첩 *dict*, 경로 = `X.Y.Z`
- 데이터 파일 4종의 역할 지도
- Stage 1 = 추출 → 필터 → 병합 → 검증
- 각 단계 **코드블록 + 줄별 설명**

▶ "19 vs 12"는 뒤(§5)에서 다룰 실제 버그의 씨앗 — 미리 기억해 두자.

핵심 숫자(이 fork 실측)

- `large_attributes.json`: **leaf 2,297**개 (=값 채우는 슬롯)
- top-level **19**개(정규 12 + 중복 7)
- 최대 깊이 5단계
- 논문 트리 **8,496** 노드 = 전 계층(중간 폴더 포함), **왼만** 값을 받음

1. 왜 트리인가 — Stage 1의 목적
2. 데이터 파일 지도
3. Stage 1 파이프라인 (코드)
4. 필터의 판정들
5. 실제 파일의 함정

1. 왜 트리인가 — Stage 1의 목적

트리는 엔진의 **조종면**(control surface)

쉽게 말하면 페르소나를 만들 때 "어떤 속성을 쓸 수 있는지"의 전체 메뉴판이 트리다. 메뉴판이 좋아야 다양·균형·깊이가 산다.

논문이 요구한 트리의 4조건 (§3.1)

① 데이터 기반 (data-driven)

수작업 나열이 아니라 실제 *ChatGPT* 대화에서 채굴.

② 계층적 (hierarchical)

큰 특질 → 세부로 자연스럽게 내려감 (x.y.z).

③ 의미 검증 (semantically-validated)

모순·중복 제거. 유사 노드 병합.

④ 진짜 개인적 (genuinely-personal)

특정 브랜드·제품 같은 과도하게 구체적 노드 배제.

한눈에 트리는 **한 번** 만들어 파일로 커밋 — 이후 모든 페르소나가 재사용(Stage 2).

2. 데이터 파일 지도

data/ 의 파일들 — 무엇이 무엇인가

파일	규모	역할
large_attributes.json	leaf 2,297	최종 트리. Stage 2가 실제로 쓰는 슬롯 참고
attributes_merged.json	leaf 4,624	병합 중간 산출물(아직 덜 정리됨)
template.json	12개	허용된 top-level 카테고리 정규 목록
occupations_english.json	932줄	직업 사전(시드용, 모듈 03)
...embeddings.pkl	2297×1536	슬롯 벡터 캐시(모듈 04)

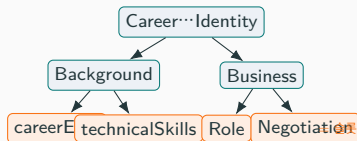
쉽게 말하면 "트리 파일" 하나가 아니라, 정리 단계별로 파일이 여러 개다. 최종본은 large_attributes.json .

▶ leaf 수는 이 fork에서 python으로 직접 카운트한 실측치.

트리 자료구조 — 중첩 dict, 빈 {} = leaf

쉽게 말하면 JSON 딕셔너리가 곧 나무다. 값이 빈 딕셔너리 {} 면 더 못 내려가는 잎(leaf)=실제 슬롯.

```
"Career and Work Identity": {  
  "Background": {  
    "careerExperience": {},  
    "technicalSkills": {}    <- leaf  
  },  
  "Business": {  
    "Role": {},              <- leaf  
    "Negotiation Skills": {} <- leaf  
  }  
}
```



깊이 최대 5. leaf만 "채울 수 있는 슬롯"이다.

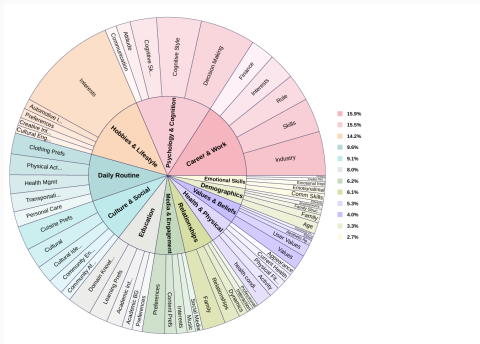
위 dict를 경로로 퍼면:

Career...Identity.Background.technicalSkills

Career...Identity.Business.Role

트리가 담은 도메인 — 균형(원본 Figure 3)

쉽게 말하면 2,297개 슬롯이 12개 대분류에 고르게 퍼져 있다 — 이 "균형"이 다양한 페르소나를 가능케 한다.



왜 재작도 대신 원본?

- 조각 수백 개 — 재현이 비효율(정책 §2.1: 원본이 더 합당하면 삽입).
- 최대 점유 **15.9%** — 단일 주제 지배 없음 (균형).

읽는 법

- 안쪽 고리 = 대분류
- 바깥 고리 = 세부(leaf 근처)

Fig. 3 (Wang et al.). 조각 크기 $\propto$ 도메인 점유율. 저작권상 **figs/** 는 커밋 제외 (로컬 전용).

3. Stage 1 파이프라인 (코드)

4단계로 트리를 짓는다 — 어느 파일이 어느 단계



쉽게 말하면 원재료(대화) → 부스러기 속성 → 걸러내고 → 비슷한 걸 합치고 → 중복 없는 경로 목록으로.

한눈에 이 4개가 논문 §3.1의 "추출 → 계층 병합 → 의미 검증 · 필터"에 그대로 대응.

① 추출 — 대화에서 X.Y.Z 속성 뽑기

```
# Requirements: Output each attribute as a 3 level structure (e.g. X.Y.Z)
# IMPORTANT: The first level ... MUST be chosen from the top-level categories.
Available Top-Level Categories: {'', '.join(self.persona_categories)}
Question: {question}
Personalization Reason: {reason}
Output Format: { "attributes": ["attributeA1.levelA2.levelA3", ...] }
```

- **입력:** 한 (질문, 이유) 쌍 — 실사용 대화 Puffin · prefeval · HiCUPID(약 6.4만 대화)에서 GPT-4.1이 골라낸 6.2만 개인화 QA.
- **제약 1:** 모든 속성은 3단계 X.Y.Z 형식. (트리 깊이를 강제)
- **제약 2:** 1단계(맨 앞)는 반드시 template.json 의 12개 카테고리 중 하나.
- **지시:** "너무 구체적이지 말고 일반적 · 상위" — 브랜드/제품 배제(조건 ④).
- **출력:** JSON {"attributes": [...]} — 다음 단계가 파싱.

▶ 여기서 나온 속성들은 아직 제각각. ②~④가 이를 하나의 트리로 정렬한다.

③ 병합 — 비슷한 형제 노드를 LLM이 합침

```
def merge_level_nodes(nodes, level, client):
    prompt = """analyze nodes at the same level and suggest merges
                based on semantic similarity ..."""
    # -> OLLM {old_name: new_name} 매핑반환
    mapping = validate_gpt_response(response_text)
    merged = process_merge_response(mapping, nodes_to_merge)
```

- **왜:** 서로 다른 대화가 `technicalSkills` / `Technical Skill` / `tech_skills` 처럼 **같은 뜻을 다른 이름**으로 만든다.
- **방법:** 트리를 `TreeNode` 로 만들고 **level(깊이)**별로 형제들을 모아 LLM에게 "합칠 것 제안" 요청.
- **반환:** {옛이름: 새이름} 매핑 → `process_merge_response()` 가 실제 노드를 합침.
- **효과:** 트리가 조밀·일관해짐(논문: "dense and hierarchical").

▶ 이 단계가 덜 돌면 top-level에 중복 variant가 남는다 — 이 fork의 19 vs 12 문제(§5).

④ 검증 — 임베딩 유사도로 중복 앞 제거

```
def check_path_similarity(path1, path2):  
    embeddings = model.encode([path1, path2])          # 문장임베딩  
    similarity = cosine_similarity([embeddings[0]], [embeddings[1]])[0][0]  
    threshold = 0.85                                   # 이값보다높으면너무 "비슷" = 중복  
    return similarity > threshold
```

- **model:** `sentence-transformers` (로컬 임베딩) — API 아님, 무료·오프라인.
- **cosine_similarity:** 두 경로 문장을 벡터로 바꿔 **방향 유사도**(1=동일, 0=무관) 계산.
- **threshold=0.85:** 경험적 컷. 넘으면 한쪽을 중복으로 간주해 제거.
- 이어서 `check_level_compatibility()` (부모-자식 계층이 말이 되는지 LLM 확인) + `convert_to_X.Y.Z.py` (트리→경로 목록 평탄화).

한눈에 여기서 쓰는 코사인 유사도 개념은 **모듈 04의 5:3:2 선택**과 똑같다 — 미리 익혀두자.

4. 필터의 판정들

② 필터 — 세 가지 LLM 판정

함수	묻는 것	탈락 예
<code>check_if_personalized</code>	이 속성이 개인 특성인가?	"날씨", "환율" 같은 중립 정보
<code>check_category</code>	1단계가 12개 중 맞는가?	엉뚱한 top-level → 교정
<code>check_last_segment</code>	leaf가 일반 범주인가?	"Nike Air Max"(특정 제품)

쉽게 말하면 추출된 부스러기 중 "개인을 설명하지 못하는 것 / 잘못 분류된 것 / 너무 구체적인 것"을 걸러낸다.

한눈에 세 판정 = 트리 4조건 중 ③의미검증 · ④진짜 개인적을 강제하는 게이트.

5. 실제 파일의 함정

실측: top-level이 12가 아니라 19

쉽게 말하면 정규 카테고리는 12개인데, 최종 파일엔 거의 같은 뜻의 카테고리가 여러 벌 남아 19개가 됐다 — 병합(③)이 top-level까지는 못 갔다.

중복 variant 실례 (같은 뜻, 다른 문자열)

- Core Values and Beliefs
- Core Values and Philosophy
- Core Values, Beliefs, Philosophy
- Core Values, Beliefs, *and* Philosophy

⇒ 하나여야 할 "가치·신념" 카테고리가 4벌. Lifestyle 3벌, Psychological 2벌 ...

항목	실측
정규 카테고리(template)	12
실제 top-level	19
leaf(슬롯) 수	2,297
최대 깊이	5

한눈에 이 "19 vs 12"가 모듈 05의 카테고리 drop 버그로 이어진다 — generate_profile.py 가 6개만 처리(study.md §13).

꼭 기억할 것

- 트리 = 중첩 *dict*, {} = leaf = 슬롯
- 경로 표기 `x.y.z` (3단계 강제)
- Stage 1 = 추출 → 필터 → 병합 → 검증
- 병합 · 검증에 임베딩 코사인 사용
- 최종본 `large_attributes.json` (2,297 leaf)

코드 지도

- ① `extract_personalized_attributes.py`
- ② `filter_personalized_attributes.py`
- ③ `merge_tree.py`
- ④ `check_leaves.py` + `convert_to_X.Y.Z.py`

남은 결함: top-level 19(중복) → 하루 버그의 씨앗.

한눈에 다음 `03_based_data` : 트리를 걷기 전에 필요한 시드 인물(앵커)을 만든다.

모듈 02 끝

다음: 03_based_data — 시드/앵커 코어 (Step 1)